

Three RSA Attacks: Håstad, Common Modulus, and Cycling

Each of the three attacks below reuses a tool already established in Lesson 1 — the Chinese Remainder Theorem, the Extended Euclidean Algorithm, and Euler’s theorem, respectively — now applied as an attack tool rather than a construction tool.

1. Håstad’s Broadcast Attack

Setup. Small public exponent (e.g. $e = 3$), same message m broadcast to 3 recipients with independent, pairwise-coprime moduli n_1, n_2, n_3 :

$$c_i = m^3 \bmod n_i, \quad i = 1, 2, 3.$$

Core insight. Three small, separately-opaque worlds ($\bmod n_1, \bmod n_2, \bmod n_3$) get stitched by CRT into one big world ($\bmod N = n_1 n_2 n_3$) — and against that much bigger modulus, m^3 might not wrap around at all. It is the “no modular wraparound” flaw from Lesson 1, resurfacing one level up.

Theorem 1. *If $m < \min(n_1, n_2, n_3)$, an eavesdropper who sees c_1, c_2, c_3 recovers m — no private key needed.*

Proof.

- CRT combines $x \equiv c_i \pmod{n_i}$ ($i = 1, 2, 3$) into a *unique* $C \bmod N = n_1 n_2 n_3$, computed directly from public data.
- m^3 also satisfies all three congruences (by definition of c_i), so by CRT’s uniqueness, $C \equiv m^3 \pmod{N}$.
- Since $m < \min n_i$, we get $m^3 < N$, so C and m^3 are both nonnegative integers $< N$ that agree $\bmod N$ — they are *exactly equal* as integers, not just congruent.
- Take an ordinary integer cube root of C .

Worked example. $n_1 = 143 = 11 \cdot 13$, $n_2 = 323 = 17 \cdot 19$, $n_3 = 667 = 23 \cdot 29$ (pairwise coprime), $m = 17$:

$$c_1 = 17^3 \bmod 143 = 51, \quad c_2 = 17^3 \bmod 323 = 68, \quad c_3 = 17^3 \bmod 667 = 244.$$

$N = n_1 n_2 n_3 = 30,808,063$. CRT-combining $(51, 68, 244)$ gives $C = 4913$. Since $17^3 = 4913 < N$, there was no wraparound, so C is the exact integer m^3 . Then $\sqrt[3]{4913} = 17 = m$, recovered with no private key.

Practical consequence. Never send the same or related message to multiple recipients under a small e without randomized padding; OAEP (Lesson 3) fixes this.

2. Common Modulus Attack

Setup. One shared modulus n , two users with coprime exponents e_1, e_2 ($\gcd(e_1, e_2) = 1$), same message encrypted under both:

$$c_1 = m^{e_1} \bmod n, \quad c_2 = m^{e_2} \bmod n.$$

Core insight. $\gcd(e_1, e_2) = 1$ means the Extended Euclidean Algorithm can write $1 = ae_1 + be_2$ — the same computation used to build the private key d . Here it is aimed at the exponents directly: combining c_1, c_2 with exponents a, b combines m^{e_1}, m^{e_2} into $m^1 = m$.

Theorem 2. *If $\gcd(e_1, e_2) = 1$, anyone knowing only (n, e_1, e_2, c_1, c_2) recovers m — no private key needed.*

Proof.

- Extended Euclid on (e_1, e_2) gives integers a, b with $ae_1 + be_2 = 1$.
- $c_1^a c_2^b \equiv (m^{e_1})^a (m^{e_2})^b \equiv m^{ae_1 + be_2} \equiv m^1 \equiv m \pmod{n}$, by the exponent laws alone.
- If (say) $b < 0$: compute $c_2^{-1} \bmod n$ first (Extended Euclid again), then raise that to $|b|$.

Worked example. $n = 3233$ ($p = 61, q = 53$), $e_1 = 17, e_2 = 7, m = 42$:

$$c_1 = 42^{17} \bmod 3233 = 2557, \quad c_2 = 42^7 \bmod 3233 = 240.$$

Extended Euclid(17, 7) gives $a = -2, b = 5$ (check: $-2 \cdot 17 + 5 \cdot 7 = 1$). Since $a < 0$: $c_1^{-1} \bmod n = 3123$. Then

$$m = (c_1^{-1})^2 \cdot c_2^5 \bmod n = 42.$$

Practical consequence. Never share a modulus across users or purposes, even with different exponents — it is broken the instant any two exponents on it are coprime, which is the common case.

3. Cycling Attack (Conceptual)

Setup. Only (n, e, c) with $c = m^e \bmod n$. Repeatedly re-encrypt: $c_1 = c^e \bmod n, c_2 = c_1^e \bmod n, \dots$ until the sequence returns to c .

Core insight. This is Euler's theorem, one level up. Repeatedly multiplying by a cycles back to 1 after $\varphi(n)$ steps; repeatedly *encrypting* (exponentiating by e) must also cycle back eventually, since encryption is invertible. The plaintext sits in plain sight, one step before the cycle closes.

Theorem 3. *Let k be the smallest positive integer with $c_k = c$. Then $m = c_{k-1}$.*

Proof.

- By induction, $c_i \equiv m^{e^{i+1}} \pmod{n}$.
- By definition of k : $m^{e^{k+1}} \equiv m^e \pmod{n}$, i.e. $e^{k+1} \equiv e \pmod{\text{ord}_n(m)}$.
- e is coprime to $\varphi(n)$ by construction, hence coprime to $\text{ord}_n(m) \mid \varphi(n)$ too, so one factor of e cancels: $e^k \equiv 1 \pmod{\text{ord}_n(m)}$.
- So $c_{k-1} \equiv m^{e^k} \equiv m^1 \equiv m \pmod{n}$ — one full lap plus one step back, the same picture as Euler's theorem in Lesson 1.

Worked example. $p = 47, q = 59 \Rightarrow n = 2773, \varphi(n) = 2668, e = 17, m = 123$:

$$c = 123^{17} \bmod 2773 = 1924.$$

Repeatedly raising to the 17th power, after 44 iterations the sequence returns to $c_{44} = 1924 = c$. One step earlier, $c_{43} = 123 = m$.

Why it stays conceptual. At real key sizes the cycle length is astronomically large and the attack is infeasible in practice. It demonstrates that RSA encryption is iteration inside a finite group, and previews why $\lambda(n)$ having a large prime factor matters — the same concern as avoiding smooth $p - 1$ (Lesson 1).

All three attacks above reuse tools from Lesson 1 — CRT, the Extended Euclidean Algorithm, and Euler's theorem — as attack tools rather than construction tools.